

CC3104 — Aprendizaje por Refuerzo

Examen Práctico: Auditoría del agente de gestión de inventario

Grupo 4 — La aproximación de función

Nombre	Carnet	Usuario Git
Edwin Jose Gabriel De Leon Garcia	22809	EJGDLG
Gustavo Adolfo Cruz Bardales	22779	G2309
Josué Emanuel Say Garcia	22801	JosueSay
Mathew Alexander Cordero Aquino	22982	donmatthiuz

8 de septiembre de 2026

Índice

1. Análisis del componente asignado	1
Metodología de medición	1
1.1. Entregable 4.1: el problema de escala	1
1.2. Entregable 4.2: interacciones que el modelo lineal no captura	3
1.3. Entregable 4.3: versión mejorada de <code>preprocess_state</code>	4
1.4. Entregable 4.4: comparación de error cuadrático medio	7
2. Respuestas a las preguntas de integración	9
2.1. Pregunta 1: Diagnóstico sistémico	9
2.2. Pregunta 2: Causa raíz	9
2.3. Pregunta 3: Plan de corrección mínimo viable	9
2.4. Pregunta 4: Compatibilidad de correcciones	9
3. Reflexión grupal	10
A. Anexo: uso de IA generativa	11

1. Análisis del componente asignado

Metodología de medición

Para poder medir el error del aproximador se necesita un objetivo de referencia. Como la función de transición del sistema es determinista, el MDP se resolvió de forma exacta por iteración de valor con $\gamma = 0,99$, obteniendo $Q^*(s, a)$ para todos los estados alcanzables. Se usó la función de recompensa *original* del sistema entregado, no una corregida.

La especificación declara 200 estados (10 niveles de inventario $\times$ 5 de vencimiento $\times$ 4 de demanda), pero `transition` devuelve `inventory + action - daily_demand`, que no es múltiplo de 10 salvo cuando la demanda lo es. La cerradura del espacio de estados bajo la función de transición contiene **4 367 estados**, no 200. El agente pasa la mayor parte del tiempo en estados que la especificación no declara.

Todos los ajustes se hacen sobre los 4367 estados alcanzables, reservando 24 estados declarados como conjunto de prueba (Sección 1.4).

1.1. Entregable 4.1: el problema de escala

Qué componente domina el gradiente

El aproximador es lineal, $Q(s, a; w) = w_a^\top \phi(s)$, y su actualización es la regla LMS / semi-gradiente:

$$w_a \leftarrow w_a + \alpha [y - w_a^\top \phi(s)] \nabla_{w_a} Q(s, a; w), \quad \nabla_{w_a} Q(s, a; w) = \phi(s).$$

El gradiente *es* el vector de características. Por lo tanto la magnitud del paso que recibe la coordenada i es proporcional a $\phi_i(s)$, y el aporte de esa coordenada a la norma del gradiente es $\phi_i(s)^2 / \|\phi(s)\|^2$. Con

$$\begin{aligned} \phi(s) &= [\text{inventory}, \text{days_to_expiry}, \text{demand_idx}], \\ \text{inventory} &\in [0, 100], \quad \text{days} \in \{1, 7, 14, 30, 60\}, \quad \text{demand_idx} \in \{0, 1, 2, 3\}, \end{aligned}$$

el promedio sobre los 200 estados declarados da:

Componente	Aporte a $\ \phi\ ^2$ (%)
inventory	74.95
days_to_expiry	24.96
demand_idx	0.09

Cuadro 1: Participación de cada característica en la norma del gradiente.

El nivel de inventario domina el gradiente con el 75 % y la demanda aporta el 0,09 %, unas 815 veces menos. La demanda determina si un producto se agota, y es la componente que menos peso tiene en el gradiente. El peso asociado a la demanda se mueve tres órdenes de magnitud más lento que el del inventario: para cuando el aproximador ha aprendido algo sobre la demanda, el resto de los pesos ya convergió a una solución que la ignora. El sesgo no es de capacidad sino de ritmo de aprendizaje efectivo.

Divergencia del entrenamiento

Para LMS con paso fijo α , la condición de convergencia en media es

$$0 < \alpha < \frac{2}{\|\phi(s)\|^2}.$$

El máximo observado es $\|\phi\|_{\text{máx}}^2 = 11\,709$ (estado (90, 60, crítico)), de donde

$$\alpha_{\text{máx}} = \frac{2}{11\,709} = 1,708 \times 10^{-4}.$$

α	Características originales	Características propuestas
10^{-2}	diverge (overflow)	converge ($\ w\ _\infty \approx 3626$)
10^{-3}	diverge (overflow)	—
$1,4 \times 10^{-4}$	converge	—
10^{-4}	converge	—
5×10^{-2}	—	converge (≈ 3710)
10^{-1}	—	converge (≈ 3720)

Cuadro 2: Estabilidad empírica del ajuste por SGD. Con las características originales, el α del código diverge; con las propuestas, α puede ser hasta 100 veces mayor sin problemas.

El código entregado usa $\alpha = 0,01$ por defecto en **update**, es decir **58,5 veces por encima de la cota de estabilidad**. Se verificó empíricamente ajustando w por SGD sobre los estados de entrenamiento:

Adicionalmente, el número de condición de la matriz de Gram $\frac{1}{N}\Phi^\top\Phi$ pasa de $\kappa = 1888$ con las características crudas a $\kappa = 26,3$ con las mismas tres características normalizadas. Como la velocidad de convergencia de un método de primer orden escala con κ , la normalización sola acelera la convergencia en aproximadamente dos órdenes de magnitud.

Defecto adicional: ausencia de intercepto

ϕ no incluye un término constante y **weights** se inicializa en cero. Por lo tanto, para el estado $(0, d, \text{bajo})$ se tiene $\phi = (0, d, 0)$, y para $(0, \cdot, \text{bajo})$ con d pequeño el vector es casi nulo: el modelo está forzado a pasar por el origen. Como se muestra en la Sección 1.4, esto por sí solo hace que el R^2 del aproximador original sea $-132,7$, o sea peor que predecir constantemente el promedio de Q^* .

Normalización propuesta por componente

Cada componente se lleva a $[0, 1]$ usando una constante que proviene del propio dominio, no de un escalado genérico:

Componente	Transformación	Justificación
inventory	$\text{inv}/100 \in [0, 1]$	100 es la capacidad real impuesta por el $\min(100, \cdot)$ de transition ; no es una constante arbitraria.
days_to_expiry	$(d - 1)/59 \in [0, 1]$ más el indicador $\mathbf{1}[d \leq 7]$	El rango declarado es $[1, 60]$. El indicador replica la forma exacta de la penalización $-10 \cdot \mathbf{1}[d \leq 7]$: sin él, ninguna combinación lineal de d puede representar un escalón.
demand_level	<i>One-hot</i> con bajo como base	El código ordinal 0, 1, 2, 3 impone dos supuestos falsos: que la demanda tiene orden lineal en el valor y que el espaciado es uniforme. La demanda real es 5, 15, 25, 40, cuyos incrementos son 10, 10, 15. Además bajo =0 anula la característica y hace indistinguibles dos estados con demanda baja.

Cuadro 3: Normalización propuesta. Con ella $\|\phi\|^2 \leq 5,93$ y $\alpha_{\text{máx}} = 0,337$, frente a $1,708 \times 10^{-4}$ del original.

1.2. Entregable 4.2: interacciones que el modelo lineal no captura

Un aproximador lineal sobre $\phi = [\text{inv}, d, \text{dem}]$ es aditivo: el efecto marginal de cada variable es constante y no depende de las otras, $\partial Q / \partial \text{inv} = w_{a,1}$ sin importar el valor de la demanda. En gestión de inventario farmacéutico eso es falso. Las dos interacciones más importantes son:

Interacción 1: inventario $\times$ demanda (riesgo de desabasto). El riesgo de *stockout* no depende del inventario ni de la demanda por separado, sino de su **cociente**: los días de cobertura

$$\text{cobertura}(s) = \frac{\text{inventory}}{\text{daily_demand}}.$$

Un inventario de 30 unidades es abundante con demanda baja (6 días de cobertura) y crítico con demanda crítica (0,75 días). Un modelo aditivo asigna a $\text{inv} = 30$ el mismo aporte al valor en ambos casos. Un cociente no es expresable como $w_1 x_1 + w_2 x_2$ para ningún (w_1, w_2) : es una imposibilidad representacional, no un ajuste imperfecto. Se agrega explícitamente como característica, saturada a 20 días y normalizada, junto con la brecha inmediata $\text{máx}(0, \text{dem} - \text{inv})/40$, que marca los estados en los que el desabasto ocurre en el paso siguiente.

Interacción 2: inventario $\times$ vencimiento $\times$ demanda (merma). Las unidades que van a vencerse son las que no alcanzarán a venderse antes del vencimiento:

$$\text{merma}(s) = \text{máx}(0, \text{inventory} - \text{daily_demand} \cdot \text{days_to_expiry}).$$

Es un producto seguido de una rectificación. Los 41 productos vencidos por semana en producción son exactamente esta cantidad, y el aproximador original no tiene ninguna característica que se le parezca. Con $d = 60$ y demanda crítica no hay merma con ningún inventario; con $d = 1$ y demanda baja casi todo el inventario se pierde. Un término aditivo en inv y otro en d no pueden distinguir esos dos casos. Se agrega también $\text{inv}_n \cdot \mathbf{1}[d \leq 7]$, que captura que el costo del inventario alto se invierte de signo cuando el producto está por vencer.

1.3. Entregable 4.3: versión mejorada de preprocess_state

El vector de características propuesto tiene 12 componentes:

Listing 1: código/preprocesamiento_mejorado.py

```
1 DEMAND_MAP = {"bajo": 5, "medio": 15, "alto": 25, "critico": 40}
2 DEMAND_ORDER = ["bajo", "medio", "alto", "critico"]
3 INVENTORY_MAX = 100.0
4 DEMAND_MAX = 40.0
5 EXPIRY_MAX = 60.0
6 EXPIRY_THRESHOLD = 7
7 SUPPLY_CAP = 20.0
8
9 FEATURE_NAMES = [
10     "bias",
11     "inv_n",
12     "inv_n2",
13     "exp_n",
14     "near_expiry",
15     "dem_medio",
16     "dem_alto",
17     "dem_critico",
18     "dias_cobertura",
```

#	Nombre	Definición y rol
0	bias	1 — intercepto; permite representar estados con ϕ casi nulo.
1	inv_n	$\text{inv}/100$
2	inv_n2	$(\text{inv}/100)^2$ — no linealidad del costo de almacenamiento y saturación en la capacidad.
3	exp_n	$(d - 1)/59$
4	near_expiry	$\mathbf{1}[d \leq 7]$ — discontinuidad exacta de reward.
5-7	dem_*	<i>one-hot</i> de demanda (medio, alto, crítico; base bajo).
8	dias_cobertura	$\min(\text{inv}/\text{dem}, 20)/20$ — interacción 1.
9	riesgo_merma	$\max(0, \text{inv} - \text{dem} \cdot d)/100$ — interacción 2.
10	inv_x_near_expiry	$\text{inv}_n \cdot \mathbf{1}[d \leq 7]$
11	brecha_stockout	$\max(0, \text{dem} - \text{inv})/40$

Cuadro 4: Características propuestas. Todas están en $[0, 1]$, de modo que $\|\phi\|^2 \leq 12$ por construcción (máximo observado: 5,93).

```

19     "riesgo_merma",
20     "inv_x_near_expiry",
21     "brecha_stockout",
22 ]
23 N_FEATURES = len(FEATURE_NAMES)
24
25
26 def _normalizar_demanda(demand_level):
27     if isinstance(demand_level, str):
28         return demand_level.replace("í", "i")
29     return demand_level
30
31
32 def preprocess_state(state):
33     inventory, days_to_expiry, demand_level = state
34     demand_level = _normalizar_demanda(demand_level)
35     daily_demand = DEMAND_MAP[demand_level]
36
37     inv = float(min(max(inventory, 0.0), INVENTORY_MAX))
38     days = float(min(max(days_to_expiry, 1.0), EXPIRY_MAX))
39
40     inv_n = inv / INVENTORY_MAX
41     exp_n = (days - 1.0) / (EXPIRY_MAX - 1.0)
42     near_expiry = 1.0 if days <= EXPIRY_THRESHOLD else 0.0
43     dias_cobertura = min(inv / daily_demand, SUPPLY_CAP) / SUPPLY_CAP
44     riesgo_merma = max(0.0, inv - daily_demand * days) / INVENTORY_MAX
45
46     features = [0.0] * N_FEATURES
47     features[0] = 1.0
48     features[1] = inv_n
49     features[2] = inv_n * inv_n
50     features[3] = exp_n
51     features[4] = near_expiry
52     features[5] = 1.0 if demand_level == "medio" else 0.0
53     features[6] = 1.0 if demand_level == "alto" else 0.0
54     features[7] = 1.0 if demand_level == "critico" else 0.0
55     features[8] = dias_cobertura
56     features[9] = riesgo_merma

```

```

57     features[10] = inv_n * near_expiry
58     features[11] = max(0.0, daily_demand - inv) / DEMAND_MAX
59     return features
60
61
62 def preprocess_state_original(state):
63     inventory, days_to_expiry, demand_level = state
64     demand_level = _normalizar_demanda(demand_level)
65     demand_idx = {"bajo": 0, "medio": 1, "alto": 2, "critico": 3}[demand_level]
66     return [float(inventory), float(days_to_expiry), float(demand_idx)]
67
68
69 class LinearApproximator:
70     def __init__(self, n_features=N_FEATURES, n_actions=6, q_init=0.0,
71                 normalize_step=True):
72         self.n_features = n_features
73         self.n_actions = n_actions
74         self.normalize_step = normalize_step
75         self.weights = [[0.0] * n_features for _ in range(n_actions)]
76         if q_init != 0.0:
77             for a in range(n_actions):
78                 self.weights[a][0] = q_init
79
80     def predict(self, features, action):
81         w = self.weights[action]
82         return sum(w[i] * features[i] for i in range(self.n_features))
83
84     def update(self, features, action, target, alpha=0.05):
85         error = target - self.predict(features, action)
86         if self.normalize_step:
87             norm2 = sum(f * f for f in features)
88             step = alpha / norm2 if norm2 > 0.0 else 0.0
89         else:
90             step = alpha
91         w = self.weights[action]
92         for i in range(self.n_features):
93             w[i] += step * error * features[i]
94         return error
95
96
97 def cota_estabilidad(features):
98     norm2 = sum(f * f for f in features)
99     return float("inf") if norm2 == 0.0 else 2.0 / norm2

```

Cambios al aproximador

1. **Paso normalizado.** `update` usa $\alpha_{\text{eff}} = \alpha / \|\phi(s)\|^2$ (regla LMS normalizada), que garantiza estabilidad para cualquier escala de entrada.
2. **Inicialización optimista opcional.** Como ahora existe el intercepto, fijar $w_{a,0} = q_{\text{init}}$ produce $Q(s, a) \approx q_{\text{init}}$ para todo s , lo que da exploración dirigida sin tocar ε . Con el ϕ original esto era imposible.
3. **Compatibilidad.** La firma de `preprocess_state` y de `predict/update` no cambia; solo cambia `n_features` de 3 a 12. El resto del sistema no requiere modificaciones.

¿Es suficiente el aproximador lineal?

Sí, para este dominio: $R^2 = 0,967$ sobre estados no vistos (entregable 4.4). Las no linealidades de este MDP son conocidas y de forma cerrada — un escalón en $d = 7$, un producto rectificado para la merma, un cociente para la cobertura y una saturación en la capacidad. Cuando la no linealidad se conoce, incluirla como característica y ajustar un modelo lineal encima conserva convergencia garantizada bajo aproximación lineal, pesos interpretables y costo de cómputo trivial.

Se requeriría aproximación no lineal si el estado se extendiera con variables cuya interacción no se conoce de antemano: estacionalidad, sustitución entre productos o demanda estocástica con correlación temporal.

El número de condición del bloque completo de 12 características es $\kappa \approx 2,1 \times 10^3$, similar al del original, porque `riesgo_merma` es cero en la mayoría de los estados y queda casi colineal con el resto. La normalización arregla la *escala* (que es lo que causa la divergencia), no elimina la colinealidad. Se mitiga con regularización ridge ($\lambda = 10^{-8}$ en los ajustes reportados) o eliminando la característica 10, que es la de menor aporte marginal.

1.4. Entregable 4.4: comparación de error cuadrático medio

Protocolo

- Objetivo: $Q^*(s, a)$ exacto por iteración de valor ($\gamma = 0,99$, tolerancia 10^{-10}) sobre los 4367 estados alcanzables.
- Conjunto de prueba: **24 estados representativos**, tomados de forma estratificada sobre los 200 estados declarados para cubrir todo el rango de inventario, los cinco niveles de vencimiento y los cuatro de demanda.
- Conjunto de entrenamiento: los 4343 estados restantes. Los estados de prueba *no* se usan para ajustar.
- Ajuste: mínimos cuadrados exacto por acción (ecuaciones normales con ridge $\lambda = 10^{-8}$). Se usa la solución óptima en lugar de SGD para que la comparación mida **capacidad de representación** y no calidad del optimizador; el efecto sobre el optimizador se reporta por separado en 4.1.
- Métrica: $\text{MSE} = \frac{1}{|S| \cdot |A|} \sum_s \sum_a (Q^*(s, a) - w_a^\top \phi(s))^2$.

Resultado global

Aproximador	MSE entren.	MSE prueba	R^2 prueba
Original (3 caract., sin intercepto)	1,635,206.0	2,759,751.7	−132.716
Original + intercepto (4 caract.)	1,041.6	990.1	0.952
Propuesto (12 caract.)	318.2	678.4	0.967

Cuadro 5: Comparación de MSE. La varianza de Q^* es $\sigma^2 = 20\,638,8$; $R^2 = 1 - \text{MSE}/\sigma^2$. Reducción del MSE de prueba respecto al original: **99,98 %**.

La fila intermedia (características originales más un intercepto) es una ablación. El salto de $2,76 \times 10^6$ a 990 se debe casi por completo al intercepto faltante, no a las características nuevas. Aislado ese efecto, las interacciones del entregable 4.2 aportan una **reducción adicional del 31,5 %** del MSE de prueba ($990 \rightarrow 678$) y suben el R^2 de 0,952 a 0,967.

El ajuste original es *peor que la predicción constante* ($R^2 < 0$). Un aproximador con R^2 negativo alimentando una política greedy $\arg \max_a Q(s, a)$ no ordena las acciones mejor que el azar en los estados donde ϕ es pequeño, que son los de inventario bajo, donde ocurren los desabastos.

Detalle por estado (24 estados de prueba)

Inv.	d	Demanda	$\max_a Q^*$	a^*	MSE original	MSE propuesto
0	1	bajo	3,767.5	50	13,610,118.3	233.3
0	14	bajo	3,826.1	50	9,319,062.9	216.8
0	60	bajo	4,174.6	50	887,390.8	653.9
10	7	medio	3,762.6	50	6,820,040.4	327.4
10	30	medio	3,961.0	50	2,497,293.7	211.5
20	1	medio	3,772.5	50	7,258,968.9	506.5
20	14	alto	3,813.7	50	2,344,597.2	105.3
20	60	alto	4,162.2	50	335,822.6	457.4
30	7	alto	3,767.6	50	2,600,258.0	982.1
30	30	crítico	3,895.1	50	2,425.2	1,067.7
40	1	crítico	3,726.0	50	1,196,949.6	264.0
40	14	crítico	3,784.5	50	210,012.0	1,988.1
50	1	bajo	3,797.5	50	6,165,322.4	711.7
50	14	bajo	3,856.0	50	3,409,680.5	2.9
50	60	bajo	4,204.5	50	69,797.3	94.4
60	7	medio	3,797.5	50	1,998,061.6	293.5
60	30	medio	3,995.9	50	146,148.3	140.5
70	1	medio	3,800.0	50	2,222,812.5	338.2
70	14	alto	3,856.0	50	121,051.8	31.5
70	60	alto	4,204.5	50	3,107,585.7	281.0
80	7	alto	3,800.0	50	177,655.3	503.7
80	30	crítico	3,993.4	50	1,300,516.0	1,923.6
90	1	crítico	3,800.0	50	444.9	3,752.1
90	14	crítico	3,858.5	50	432,025.0	1,194.6

Cuadro 6: MSE por estado, promediado sobre las 6 acciones.

El aproximador propuesto gana en 23 de los 24 estados.

Lectura de la tabla

- El aproximador propuesto gana en **23 de 24** estados. El único estado donde pierde es (90, 1, crítico), con inventario máximo y vencimiento inmediato: ahí $\|\phi_{\text{orig}}\|$ es grande y el modelo sin intercepto acierta la escala de Q^* por coincidencia de magnitud.
- El error del original crece a medida que el inventario baja (1.4×10^7 en $\text{inv} = 0$), exactamente el efecto del intercepto ausente descrito en 4.1.
- El error residual del modelo propuesto se concentra en los estados de demanda crítica con vencimiento cercano ((90, 1, crítico): 3752). Son los estados de frontera del dominio y los peor cubiertos por el conjunto de entrenamiento.

- La acción óptima es **pedir 50** en los 24 estados de prueba. Con la función de recompensa original, pedir el máximo *sí* es la política óptima del MDP: el “94 % de estados con **pedir_50**” de las métricas de entrenamiento no es un fallo de aprendizaje, sino el resultado de una recompensa mal especificada. Ninguna corrección del preprocesamiento lo modifica.

Reproducibilidad

Listing 2: Reproducción de los resultados

```
python3 codigo/experimento_mse.py
```

No requiere dependencias externas (Python puro). Los valores crudos, incluidos los pesos ajustados, quedan en **resultados/resultados_44.json**.

2. Respuestas a las preguntas de integración

2.1. Pregunta 1: Diagnóstico sistémico

2.2. Pregunta 2: Causa raíz

2.3. Pregunta 3: Plan de corrección mínimo viable

2.4. Pregunta 4: Compatibilidad de correcciones

3. Reflexión grupal

A. Anexo: uso de IA generativa

Se usó IA generativa como apoyo de herramienta, no para el análisis del componente. A continuación los prompts empleados y la respuesta que se obtuvo de cada uno.

Prompt 1: compilación del documento

No tengo pdflatex instalado en el sistema y estoy en NixOS. Necesito compilar informe_grupo4.tex a PDF sin instalar nada de forma permanente. El documento usa babel en español, booktabs, longtable, siunitx, enumitem y listings con lsteinputlisting. Dame el comando exacto con nix-shell -p, indicando qué esquema de TeX Live necesito para que no falte ningún paquete, y explícame por qué hay que correr pdflatex más de una vez.

Respuesta obtenida:

```
nix-shell -p texlive.combined.scheme-full --run \
'pdflatex -interaction=nonstopmode -halt-on-error informe_grupo4.tex && \
pdflatex -interaction=nonstopmode -halt-on-error informe_grupo4.tex'
```

Prompt 2: escritura de las fórmulas

Convierte estas expresiones a $\text{\LaTeX}$ con amsmath, en modo display y tipográficamente correctas: (1) la regla de actualización semi-gradiente para un aproximador lineal $Q(s, a; w) = w_a^\top \phi(s)$, dejando explícito que el gradiente respecto a w_a es $\phi(s)$; (2) la condición de estabilidad de LMS $0 < \alpha < 2/\|\phi(s)\|^2$; (3) el error cuadrático medio promediado sobre estados y acciones. Requisitos: usa $\text{\LaTeX}$ y no $||$; nombres de variables del código en $\text{\LaTeX}$ dentro de modo matemático; y si una línea se pasa del margen, parte la ecuación con gather* en vez de dejar overfull hbox.

Respuesta obtenida:

Regla de actualización semi-gradiente:

$$w_a \leftarrow w_a + \alpha [y - w_a^\top \phi(s)] \nabla_{w_a} Q(s, a; w), \quad \nabla_{w_a} Q(s, a; w) = \phi(s).$$

Condición de estabilidad de LMS:

$$0 < \alpha < \frac{2}{\|\phi(s)\|^2}.$$

Error cuadrático medio sobre estados y acciones:

$$\text{MSE} = \frac{1}{|S| \cdot |A|} \sum_s \sum_a (Q^*(s, a) - w_a^\top \phi(s))^2.$$